



INSTITUTO SUPERIOR POLITÉCNICO GAYA

Licenciatura em Engenharia Informática

Projeto de Engenharia Informática em Contexto Empresarial

BoxToCar

SISTEMA DE PICKING HUMANO-MÁQUINA: INDÚSTRIA 5.0

Luís Miguel Marques Rafael

2023103849

*Relatório de estágio/projeto orientado pelo Professor Fernando Luís Ferreira de Almeida
e apresentado à Escola Superior de Ciência e Tecnologia*

Vila Nova de Gaia

Junho de 2026

Dedicatória

*À minha família, pelo apoio incondicional
e por acreditarem em mim em cada etapa deste percurso.
Aos meus amigos, pela presença nos momentos certos.*

Agradecimentos

Agradeço, em primeiro lugar, à minha família, por todos os valores e ensinamentos transmitidos e pelo apoio constante ao longo de todo o percurso académico.

Ao ISPGAYA e, em particular, ao Professor Fernando Luís Ferreira de Almeida, orientador deste projeto, pela disponibilidade, orientação e rigor com que acompanhou o trabalho.

A todos os docentes da Licenciatura em Engenharia Informática, pelo contributo determinante na minha formação, e aos colegas, hoje amigos, que tornaram esta jornada mais leve e memorável.

Resumo

O presente relatório descreve o desenvolvimento do BoxToCar, uma prova de conceito de um sistema de picking colaborativo humano-máquina inscrito no paradigma da Indústria 5.0, realizado no âmbito do projeto final da Licenciatura em Engenharia Informática. Ao contrário das abordagens que procuram substituir o operador, o sistema explora a complementaridade entre o ser humano — responsável pela destreza da recolha — e um robô móvel virtual (AMR) — responsável pelo transporte da carga —, coordenados por um núcleo lógico que os faz convergir em pontos de encontro eficientes.

A solução é composta por três componentes integrados: uma API REST (Node.js/Express) que concentra a lógica de negócio e a coordenação; um backoffice web para o gestor, com um mapa interativo do armazém, importação de plantas, gestão de stock e visualização da coordenação; e uma aplicação móvel (Flutter) que guia o operador até à prateleira e, em seguida, até ao ponto de encontro com o robô. O cálculo dos pontos de encontro e das rotas recorre a algoritmos exatos de procura em grelha (BFS), minimizando a distância percorrida pelo operador com carga.

O sistema foi validado através de testes funcionais às operações críticas e de testes ao algoritmo de coordenação, cujas métricas confirmaram uma redução média de cerca de 73% na distância que o operador percorre com carga, nos cenários testados. A arquitetura assenta em autenticação por tokens JWT, palavras-passe protegidas por bcrypt e isolamento de dados entre armazéns.

Palavras-chave: Indústria 5.0; Picking Colaborativo; Robôs Móveis Autónomos; Ponto de Encontro; Algoritmos de Rota; API REST; Flutter; Coordenação Humano-Máquina.

Abstract

This report describes the development of BoxToCar, a proof of concept of a collaborative human-machine picking system framed within the Industry 5.0 paradigm, carried out as the final project of the Bachelor's Degree in Computer Engineering. Rather than replacing the operator, the system exploits the complementarity between the human — responsible for the dexterity of picking — and a virtual autonomous mobile robot (AMR) — responsible for transporting the load —, coordinated by a logical coordination core that makes them converge at efficient meeting points.

The solution comprises three integrated components: a REST API (Node.js/Express) concentrating the business logic and coordination; a web backoffice for the manager, with an interactive warehouse map, layout import, stock management and coordination visualisation; and a mobile application (Flutter) that guides the operator to the shelf and then to the meeting point with the robot. The computation of meeting points and routes relies on exact grid path-finding algorithms (BFS), minimising the distance travelled by the operator while carrying a load.

The system was validated through functional tests of the critical operations and through tests of the coordination algorithm, whose metrics confirmed an average reduction of about 73% in the operator's loaded travel distance, in the tested scenarios. The architecture relies on JWT token authentication, passwords protected with bcrypt, and data isolation between warehouses.

Keywords: Industry 5.0; Collaborative Picking; Autonomous Mobile Robots; Meeting Point; Routing Algorithms; REST API; Flutter; Human-Machine Coordination.

Lista de Abreviaturas e Siglas

- AGV** — Automated Guided Vehicle (Veículo de Guiado Automático)
- AMR** — Autonomous Mobile Robot (Robô Móvel Autônomo)
- API** — Application Programming Interface
- APA** — American Psychological Association
- BFS** — Breadth-First Search (Procura em Largura)
- ERP** — Enterprise Resource Planning
- HTTP** — HyperText Transfer Protocol
- ISPGAYA** — Instituto Superior Politécnico Gaya
- JSON** — JavaScript Object Notation
- JWT** — JSON Web Token
- PME** — Pequena e Média Empresa
- REST** — Representational State Transfer
- SGBD** — Sistema de Gestão de Bases de Dados
- SKU** — Stock Keeping Unit
- SQL** — Structured Query Language
- WMS** — Warehouse Management System (Sistema de Gestão de Armazém)

Índice

Dedicatória	2
Agradecimentos	3
Resumo	4
Abstract	5
Lista de Abreviaturas e Siglas	6
Índice.....	7
Índice de Apêndices	10
Índice de Figuras.....	11
Índice de Tabelas	12
1. Introdução	13
1.1. Contexto e motivação	13
1.2. Objetivos.....	13
1.3. Estrutura do documento	14
2. Caracterização da Instituição	15
3. Enquadramento e Análise de Necessidades.....	16
3.1. Contexto Institucional	16
3.2. Análise de Necessidades	16
4. Análise de Soluções Disponíveis no Mercado	17
4.1. Sistemas de gestão de armazém tradicionais	17
4.2. Sistemas colaborativos e robôs móveis autónomos	17
4.3. Comparação de soluções	17
4.4. Limitações e justificação do desenvolvimento próprio	17
5. Bases Teóricas.....	19
5.1. Indústria 5.0 e a colaboração humano-máquina.....	19
5.2. Robôs móveis autónomos (AMR) na intralogística	19
5.3. Sistemas WMS e o processo de picking.....	19
5.4. Estratégias de picking colaborativo	19
5.5. Arquitetura cliente-servidor e APIs REST	20
5.6. Algoritmos de procura de caminho em grelha	20
5.7. O problema do ponto de encontro (rendezvous)	20
5.8. Autenticação (JWT) e hashing de palavras-passe (bcrypt)	21
5.9. Persistência relacional e desenvolvimento multiplataforma.....	21
6. Análise, Arquitetura e Planeamento	22
6.1. Requisitos do sistema.....	22
6.1.1. Requisitos não funcionais	22

6.2. Arquitetura da solução	22
6.3. Modelo de dados	23
6.4. Planeamento e metodologia	24
7. Manual de Utilizador.....	25
7.1. Introdução.....	25
7.2. Guia do Gestor (Backoffice Web)	25
7.2.1. Criar conta e armazém.....	25
7.2.2. Iniciar sessão	25
7.2.3. O painel e o mapa do armazém	25
7.2.4. Definir o tamanho e construir prateleiras	25
7.2.5. Dar entrada de produtos	25
7.2.6. Gerar ordens de picking	25
7.2.7. Planear a coordenação humano-máquina	26
7.3. Guia do Operador (Aplicação Móvel).....	26
7.3.1. Criar conta e iniciar sessão	26
7.3.2. Executar uma recolha	26
7.4. Resolução de Problemas (FAQ)	26
8. Autenticação, Autorização e Multi-armazém	28
9. Gestão do Armazém e Mapa Interativo	29
10. Gestão de Stock e Ordens de Picking.....	30
11. Coordenação Humano-Máquina	32
11.1. Modelo de coordenação	32
11.2. Cálculo do ponto de encontro	32
11.3. Algoritmo	32
11.4. Implementação	32
11.5. Visualização.....	33
11.6. Métricas de eficiência	33
12. Aplicação Móvel do Operador	34
13. Testes de Validação	35
13.1. Metodologia de teste	35
13.2. Testes funcionais	35
13.3. Testes ao algoritmo de coordenação.....	35
13.4. Testes unitários ao módulo de coordenação	36
13.5. Análise dos resultados.....	36
14. Conclusões	37
14.1. Resultados obtidos	37
14.2. Dificuldades e soluções	37

14.3. Melhorias futuras	37
14.4. Limitações.....	37
14.5. Contributo pessoal e aprendizagens	38
Referências Bibliográficas.....	39
Glossário	41
Apêndices	42
Apêndice A — Estrutura de rotas da API.....	42
Apêndice B — Esquema das tabelas da base de dados	42
Apêndice C — Pseudocódigo do algoritmo de coordenação.....	42

Índice de Apêndices

Apêndice A — Estrutura de rotas da API

Apêndice B — Esquema das tabelas da base de dados

Apêndice C — Pseudocódigo do algoritmo de coordenação

Apêndice D — Declaração de utilização de ferramentas de inteligência artificial

Índice de Figuras

Figura 1 - Arquitetura cliente-servidor da solução.....	23
Figura 2 - Modelo de dados (diagrama entidade-relação).....	23
Figura 3 - Ecrã de início de sessão do backoffice web.	28
Figura 4 - Backoffice web com o mapa interativo do armazém.....	29
Figura 5 - Entrada de produtos e simulador de ordens de picking (ERP).	30
Figura 6 - Fluxo do processo de picking.....	31
Figura 7 - Coordenação humano-máquina: ponto de encontro e rotas calculadas. .	33
Figura 8 - Aplicação móvel do operador: tarefa e ponto de encontro com o robô. ...	34

Índice de Tabelas

Tabela 1 - Comparação entre soluções de mercado e o BoxToCar.	17
Tabela 2 - Requisitos funcionais e respetiva prioridade.	22
Tabela 3 - Tabelas principais da base de dados.	24
Tabela 4 - Casos de teste funcionais e resultados obtidos.	35
Tabela 5 - Resultados do algoritmo de coordenação (armazém 7×6).	36
Tabela 6 - Resultados do algoritmo de coordenação num segundo layout (5×7). ...	36

1. Introdução

1.1. Contexto e motivação

A quarta revolução industrial (Indústria 4.0) centrou-se na automação, na conectividade e na digitalização dos processos produtivos. Mais recentemente, o conceito de Indústria 5.0 tem vindo a complementar essa visão, recolocando o ser humano no centro do sistema produtivo e valorizando a colaboração — e não a substituição — entre pessoas e máquinas, com objetivos de resiliência, sustentabilidade e bem-estar do trabalhador (Xu et al., 2021; Leng et al., 2022).

A intralogística, e em particular a operação de armazém, é um domínio onde este paradigma tem aplicação direta. O picking — a recolha física dos artigos que compõem uma encomenda — é reconhecido como uma das operações mais críticas e dispendiosas de um armazém, sobretudo pelo tempo despendido em deslocações e pelo esforço físico associado ao transporte de carga (Van Gils et al., 2018; Boysen et al., 2019). A introdução de robôs móveis autónomos (AMR) na intralogística tem-se revelado uma via eficaz para aumentar a produtividade e reduzir o esforço humano (Fragapane et al., 2021).

É neste contexto que se enquadra o BoxToCar. Em vez de automatizar totalmente a recolha — o que exigiria manipuladores complexos e dispendiosos —, o sistema adota uma abordagem colaborativa: o operador humano executa a recolha de precisão (destreza), enquanto um robô virtual assume o transporte da carga ao longo de rotas fixas, encontrando-se com o operador em pontos estratégicos. A motivação do projeto reside em demonstrar, através de uma prova de conceito funcional, a viabilidade e o valor deste modelo de coordenação humano-máquina.

1.2. Objetivos

Em conformidade com a proposta de projeto, o objetivo central é o desenvolvimento de uma prova de conceito de um sistema logístico colaborativo (Indústria 5.0) que coordene, de forma lógica, um operador humano e um robô virtual na preparação de encomendas. Foram definidos os seguintes objetivos específicos:

- Desenvolver um protótipo de software composto por Backend (API), Backoffice Web e Aplicação Móvel.
- Implementar a importação dinâmica de layouts de armazém em grelha 2D através de ficheiros estruturados (JSON).
- Criar um sistema lógico de coordenação que sincronize as tarefas de recolha do operador com o transporte de um robô virtual (AMR).

- Aplicar algoritmos exatos de cálculo de rotas para definir pontos de encontro eficientes entre o humano e a máquina, reduzindo a distância percorrida pelo operador com carga.
- Garantir a segurança do sistema (autenticação, cifragem) e o isolamento de dados entre múltiplos armazéns.

1.3. Estrutura do documento

Para além desta introdução, o documento organiza-se em mais treze capítulos. Os capítulos 2 e 3 apresentam o enquadramento institucional e a análise de necessidades. O capítulo 4 analisa as soluções de mercado. O capítulo 5 desenvolve, com profundidade, os fundamentos teóricos (Indústria 5.0, AMR, WMS, algoritmos de rota e ponto de encontro). O capítulo 6 descreve a análise, a arquitetura e o planeamento. O capítulo 7 é um manual de utilizador do sistema. Os capítulos 8 a 12 detalham a implementação das várias componentes, com destaque para o capítulo 11, dedicado à coordenação humano-máquina. O capítulo 13 apresenta os testes de validação e o capítulo 14 as conclusões. Seguem-se as referências, o glossário e os apêndices.

1.4. Declaração de utilização de ferramentas de inteligência artificial

No desenvolvimento deste projeto foram utilizadas ferramentas de assistência baseadas em inteligência artificial, cuja utilização se declara para efeitos de transparência.

O apoio incidiu sobre a utilização do sistema de controlo de versões Git e da plataforma GitHub, com os quais o autor tinha pouca experiência prévia, bem como sobre a revisão crítica, a correção linguística e a formatação do texto deste relatório, incluindo a verificação das referências bibliográficas segundo as normas APA e a construção da página web do projeto.

A conceção do sistema, a definição do algoritmo de coordenação humano-máquina, o desenho da arquitetura e do modelo de dados, a implementação e a validação experimental dos resultados são da inteira responsabilidade do autor. Todo o conteúdo produzido com o apoio destas ferramentas foi revisto e verificado antes de ser incorporado no relatório.

O detalhe das ferramentas utilizadas, das respetivas versões, dos objetivos, dos prompts submetidos e dos resultados obtidos encontra-se descrito no Apêndice D.

2. Caracterização da Instituição

O projeto BoxToCar foi desenvolvido no âmbito da unidade curricular de Projeto de Engenharia Informática em Contexto Empresarial, do 3.º ano da Licenciatura em Engenharia Informática do Instituto Superior Politécnico Gaya (ISPGAYA), entidade promotora e acolhedora do trabalho. O ISPGAYA é uma instituição de ensino superior sediada em Vila Nova de Gaia, vocacionada para a formação técnica e científica nas áreas da engenharia e da tecnologia, com forte ligação ao tecido empresarial e à aplicação prática do conhecimento.

O cenário de aplicação considerado é o de uma organização com um ou mais armazéns, na qual um gestor administra o espaço e o stock e um conjunto de operadores executa as recolhas, apoiados por robôs móveis de transporte. Esta caracterização orientou tanto a modelação dos dados como o desenho das interfaces e da lógica de coordenação.

O domínio escolhido — o picking colaborativo — revelou-se particularmente rico do ponto de vista técnico, por reunir num único sistema o desenho de uma API, a modelação de uma base de dados, a construção de interfaces web e móveis, a implementação de mecanismos de segurança e, sobretudo, a aplicação de algoritmos de optimização de rotas e coordenação.

3. Enquadramento e Análise de Necessidades

3.1. Contexto Institucional

A unidade curricular de Projeto tem como propósito a aplicação integrada dos conhecimentos da licenciatura na conceção de uma solução de software completa, da análise de requisitos à validação. O BoxToCar inscreve-se numa linha de trabalhos orientados para a resolução de problemas reais de intralogística, procurando materializar, num protótipo, os princípios de colaboração humano-máquina que caracterizam a Indústria 5.0.

3.2. Análise de Necessidades

A recolha de encomendas em armazém combina duas exigências distintas: a destreza para localizar e manusear artigos, na qual o ser humano é insuperável, e o transporte de carga por longas distâncias, tarefa fisicamente exigente e propensa a fadiga e lesões músculo-esqueléticas (Gualtieri et al., 2021). As soluções tradicionais tendem a colocar todo o esforço no operador, que percorre repetidamente longos trajetos a empurrar carros de carga.

Da análise destas necessidades resulta a oportunidade para um sistema que separe estas duas responsabilidades: manter o operador na tarefa de precisão e delegar o transporte pesado a um robô, coordenando ambos de forma a que a distância percorrida com carga pelo humano seja mínima. É a esta necessidade — reduzir a distância que o operador percorre com carga sem comprometer a flexibilidade da recolha manual — que o BoxToCar procura responder.

4. Análise de Soluções Disponíveis no Mercado

4.1. Sistemas de gestão de armazém tradicionais

As soluções de gestão de armazém mais difundidas surgem como módulos de sistemas ERP (por exemplo, SAP EWM ou Odoo) ou como plataformas WMS especializadas. Oferecem funcionalidades abrangentes de inventário, localização, expedição e relatórios, sendo adequadas a operações de média e grande dimensão, mas caracterizam-se por custos de licenciamento elevados e complexidade de configuração significativa (Boysen et al., 2019).

4.2. Sistemas colaborativos e robôs móveis autónomos

Uma segunda geração de soluções integra robôs móveis autónomos no fluxo de picking. Modelos como o goods-to-person (o robô traz a prateleira ao operador, popularizado pela Kiva/Amazon Robotics) ou o picker-following (o robô acompanha o operador) demonstram ganhos de produtividade assinaláveis (Azadeh et al., 2019; Fragapane et al., 2021). Estas soluções são, contudo, dispendiosas e assentam em frotas físicas com navegação autónoma e prevenção de colisões em tempo real.

Entre as variantes documentadas contam-se ainda os pontos de consolidação, em que o operador deposita os artigos recolhidos em locais de transferência a partir dos quais o robô assume o transporte — modelo de que a abordagem deste projeto se aproxima (Azadeh et al., 2019).

4.3. Comparação de soluções

A Tabela 1 compara, de forma sintética, estas categorias com a abordagem do BoxToCar, considerando critérios relevantes para um contexto de PME ou de prova de conceito académica.

Critério	WMS/ERP tradicional	Sistemas com AMR (mercado)	BoxToCar
Custo	Elevado	Muito elevado	Nulo (protótipo próprio)
Complexidade	Alta	Muito alta	Baixa
Colaboração humano-máquina	Inexistente	Sim (frota física)	Sim (robô virtual)
Otimização de ponto de encontro	Não	Parcial/proprietária	Sim (algoritmo exato)
Adequação a PME/académico	Baixa	Baixa	Alta

Tabela 1 - Comparação entre soluções de mercado e o BoxToCar.

4.4. Limitações e justificação do desenvolvimento próprio

As soluções de mercado, embora robustas, são inacessíveis a organizações de pequena dimensão e, por serem produtos fechados, não permitem explorar nem demonstrar os mecanismos internos de coordenação. Conforme referido na proposta, a componente robótica do BoxToCar é puramente virtual e opera em rotas fixas e previsíveis, o que permite demonstrar a lógica de um sistema colaborativo moderno sem incorrer na complexidade da navegação autónoma e da prevenção de colisões em tempo real. Justifica-se, assim, o desenvolvimento próprio como via para compreender, controlar e validar toda a cadeia — da base de dados ao algoritmo de coordenação.

5. Bases Teóricas

Este capítulo aprofunda os fundamentos que sustentam as opções técnicas do projeto, desde o paradigma da Indústria 5.0 e os robôs móveis autônomos até aos algoritmos de procura de caminho e ao problema do ponto de encontro.

5.1. Indústria 5.0 e a colaboração humano-máquina

A Indústria 5.0 é frequentemente descrita como uma evolução, e não uma rutura, face à Indústria 4.0: mantém a base tecnológica (automação, dados, conectividade) mas acrescenta-lhe uma orientação humano-cêntrica, sustentável e resiliente (Xu et al., 2021; Leng et al., 2022). Um dos seus pilares é a colaboração entre humanos e máquinas, na qual as capacidades complementares de ambos são exploradas: a flexibilidade cognitiva e a destreza do ser humano, e a força, a repetibilidade e a resistência da máquina (Maddikunta et al., 2022). Aplicado ao picking, este princípio traduz-se em atribuir ao operador a recolha de precisão e ao robô o transporte de carga, reduzindo a fadiga e o risco de lesões associados ao esforço físico repetido (Gualtieri et al., 2021).

5.2. Robôs móveis autônomos (AMR) na intralogística

Os robôs móveis autônomos (AMR) distinguem-se dos tradicionais veículos de guiado automático (AGV) pela sua capacidade de decisão e navegação flexível; contudo, em ambientes estruturados, é frequente — e vantajoso — restringir a sua circulação a corredores principais e rotas fixas, aumentando a previsibilidade e a segurança (Fragapane et al., 2021). No BoxToCar, o robô é modelado como um AMR virtual que circula exclusivamente pelo corredor principal (o perímetro da grelha), o que dispensa a modelação de navegação autónoma e de prevenção de colisões, mantendo o foco na lógica de coordenação.

5.3. Sistemas WMS e o processo de picking

Um sistema de gestão de armazém (WMS) apoia e controla as operações de receção, armazenamento, localização e expedição, procurando maximizar a utilização do espaço e a exatidão do inventário. Dentro destas operações, o picking é aquela em que a deslocação tem maior peso; a literatura recente sobre desenho de sistemas de picking sublinha precisamente a importância de reduzir as distâncias percorridas e de coordenar recursos (Van Gils et al., 2018; Boysen et al., 2019). O BoxToCar atua sobre esta dimensão, minimizando a distância que o operador percorre com carga.

5.4. Estratégias de picking colaborativo

Entre as estratégias de picking assistido por robôs contam-se o goods-to-person, o picker-following e os pontos de consolidação, nos quais o operador deposita os

artigos recolhidos em pontos de transferência a partir dos quais o robô assume o transporte (Azadeh et al., 2019). O modelo adotado neste projeto aproxima-se deste último: define-se, para cada recolha, um ponto de encontro no corredor principal onde o operador entrega a carga ao robô, minimizando o percurso manual com peso.

5.5. Arquitetura cliente-servidor e APIs REST

A solução adota uma arquitetura cliente-servidor, na qual um servidor central concentra a lógica de negócio e o acesso aos dados, e vários clientes (backoffice web e aplicação móvel) consomem os seus serviços. A comunicação segue o estilo arquitetural REST, proposto por Fielding (2000), que utiliza os métodos do protocolo HTTP e a troca de representações em JSON, promovendo a separação de responsabilidades, a ausência de estado no servidor e a reutilização da mesma lógica por diferentes clientes.

5.6. Algoritmos de procura de caminho em grelha

O cálculo de rotas num armazém modelado como grelha 2D é um problema clássico de procura de caminho mais curto num grafo. Três algoritmos são de referência (Cormen et al., 2009): a procura em largura (Breadth-First Search, BFS), que garante o caminho mais curto em número de passos quando as arestas têm custo unitário; o algoritmo de Dijkstra, que generaliza a BFS para arestas com custos não negativos; e o algoritmo A*, que acelera a procura recorrendo a uma heurística admissível (por exemplo, a distância de Manhattan) para orientar a expansão (Hart et al., 1968). Numa grelha uniforme, na qual todos os movimentos têm custo unitário, a BFS é exata e suficiente, pelo que foi a estratégia adotada. Caso se introduzam custos diferenciados — por exemplo, corredores de trânsito mais rápido —, a BFS deixa de garantir o caminho de menor custo, sendo o algoritmo de Dijkstra o adequado; o algoritmo A* acrescenta-lhe uma heurística admissível, que reduz o número de células exploradas sem comprometer a otimalidade da solução. Importa notar que uma simples distância geométrica (como a distância de Manhattan até ao corredor) seria insuficiente: as prateleiras são obstáculos e uma medida em linha reta ignoraria as "paredes", conduzindo a caminhos inválidos. É precisamente a presença de obstáculos que obriga a uma procura sobre a grelha, em vez de uma fórmula fechada.

5.7. O problema do ponto de encontro (rendezvous)

A coordenação entre o operador e o robô configura uma variante do problema do ponto de encontro (rendezvous): dados dois agentes com origens distintas, determinar um ponto de convergência que optimize um dado critério. Neste projeto, o critério é assimétrico — pretende-se minimizar a distância percorrida pelo operador com carga (o recurso mais oneroso e sujeito a fadiga), aceitando que o robô, que não se cansa, percorra distâncias maiores. O ponto de encontro é, por isso, a célula do

corredor principal mais próxima da prateleira, obtida por uma procura em largura a partir dos acessos à prateleira.

5.8. Autenticação (JWT) e hashing de palavras-passe (bcrypt)

A autenticação baseia-se em JSON Web Tokens (JWT), um formato compacto e autossuficiente definido na RFC 7519 (Jones et al., 2015): após validar as credenciais, o servidor emite um token assinado que o cliente envia em cada pedido, permitindo autorização sem estado de sessão. As palavras-passe são protegidas com a função de hashing bcrypt (Provos & Mazières, 1999), que aplica um fator de custo configurável e um salt único por registo, tornando inviável a sua recuperação em caso de fuga.

5.9. Persistência relacional e desenvolvimento multiplataforma

A persistência é assegurada por um sistema de gestão de bases de dados relacional (MySQL), que organiza a informação em tabelas relacionadas por chaves, com consultas parametrizadas que previnem injeções de SQL e um pool de ligações para desempenho. A única exceção é a cláusula LIMIT de uma operação de baixa de stock que, por não ser parametrizável, recebe um valor previamente convertido para inteiro, evitando a concatenação direta de texto não validado (limitação retomada na secção 14.4). A aplicação do operador foi desenvolvida em Flutter, framework que, a partir de uma única base de código em Dart, gera aplicações para várias plataformas, reduzindo o esforço de desenvolvimento (Google, 2023).

6. Análise, Arquitetura e Planejamento

6.1. Requisitos do sistema

Os requisitos funcionais foram derivados diretamente dos objetivos da proposta e encontram-se, com a respectiva prioridade, na Tabela 2.

Ref.	Requisito funcional (conforme a proposta)	Prioridade
RF1	Protótipo com Backend (API), Backoffice Web e Aplicação Móvel	Alta
RF2	Importação dinâmica de layouts de armazém em grelha 2D (JSON)	Alta
RF3	Autenticação e isolamento de dados por armazém	Alta
RF4	Gestão de prateleiras, stock e ordens de picking	Alta
RF5	Coordenação lógica operador–robô com ponto de encontro	Alta
RF6	Cálculo de rotas por algoritmo exato e métricas de eficiência	Alta
RF7	Guiar o operador (app) até à prateleira e ao ponto de encontro	Alta

Tabela 2 - Requisitos funcionais e respetiva prioridade.

6.1.1. Requisitos não funcionais

- Segurança — palavras-passe protegidas por hashing, rotas sensíveis protegidas e dados isolados por armazém.
- Usabilidade — interfaces claras e consistentes entre o backoffice web e a aplicação móvel.
- Portabilidade — aplicação do operador funcional em várias plataformas a partir de uma base de código.
- Desempenho — cálculo de rotas em tempo interativo; pool de ligações à base de dados.
- Manutenibilidade — separação da lógica de coordenação num módulo próprio e testável.

6.2. Arquitetura da solução

O sistema é constituído por três componentes (Figura 1). O backend (Node.js/Express) constitui o núcleo do sistema: expõe a API REST, comunica com a base de dados e concentra a lógica de coordenação. O backoffice web (HTML, CSS e JavaScript), servido pelo backend, disponibiliza o mapa interativo e a visualização da coordenação. A aplicação móvel (Flutter) guia o operador. Os clientes comunicam com a mesma API, partilhando dados e regras de negócio.

Figura — Arquitetura da solucao (cliente-servidor)

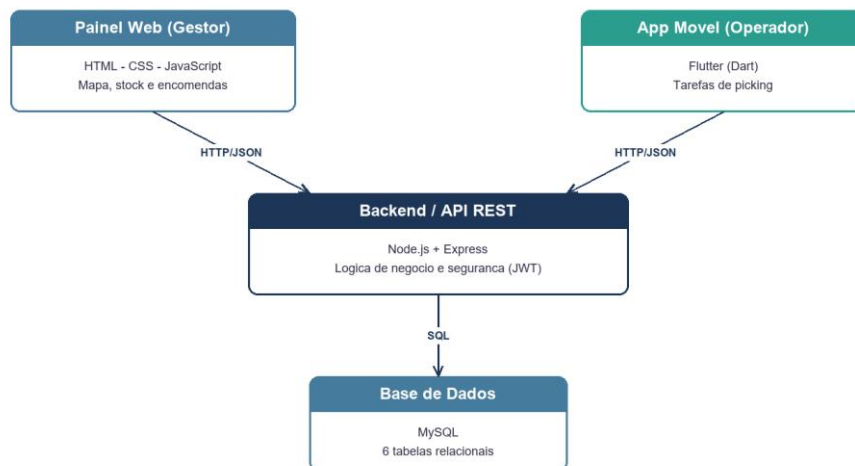
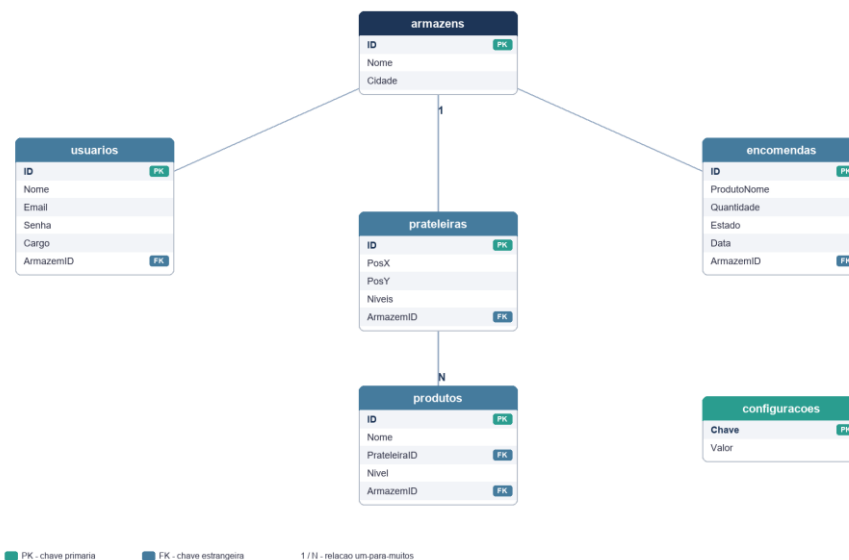


Figura 1 - Arquitetura cliente-servidor da solução.

6.3. Modelo de dados

A base de dados é composta por seis tabelas principais (Figura 2 e Tabela 3). Cada produto corresponde a uma unidade física de stock, o que permite calcular o nível de stock de um artigo contando os registos correspondentes.

Figura — Modelo de dados (Diagrama Entidade-Relacao)



PK - chave primaria FK - chave estrangeira 1 / N - relacao um para muitos

Figura 2 - Modelo de dados (diagrama entidade-relação).

Tabela	Descrição
armazens	Cada armazém (nome e cidade).
usuarios	Contas de gestor/operador e armazém associado.
prateleiras	Posição (X,Y) e níveis de cada estrutura.
produtos	Cada registo é uma unidade física de stock, associada a uma prateleira.

Tabela	Descrição
encomendas	Ordens de picking e respetivo estado.
configuracoes	Parâmetros do armazém (dimensões da grelha).

Tabela 3 - Tabelas principais da base de dados.

O modelo adota algumas simplificações conscientes, próprias de uma prova de conceito. Em primeiro lugar, cada unidade de stock é um registo distinto na tabela produtos, em vez de uma coluna de quantidade; esta opção facilita a leitura e a demonstração, mas não escala para grandes volumes. Em segundo lugar, as encomendas referenciam o produto pelo nome (ProdutoNome), e não por uma chave estrangeira, por simplicidade na ligação ao simulador ERP, o que torna a associação frágil caso existam nomes repetidos. Por fim, a tabela configuracoes assume, no protótipo, um único armazém por gestor. Estas opções são retomadas nas limitações (secção 14.4).

6.4. Planeamento e metodologia

O desenvolvimento seguiu o modelo de desenvolvimento incremental (Sommerville, 2016), no qual o sistema é construído por uma sucessão de incrementos, cada um acrescentando um conjunto reduzido de funcionalidades a uma versão já operacional. Esta abordagem insere-se na tradição do desenvolvimento iterativo e incremental, cuja utilização em engenharia de software é anterior aos métodos ágeis (Larman & Basili, 2003).

A escolha deste modelo, em vez de um processo sequencial, deveu-se ao facto de os requisitos terem sido clarificados à medida que o sistema tomava forma — em particular o algoritmo de coordenação, cujo critério de escolha do ponto de encontro só estabilizou após as primeiras experiências sobre a grelha. Optou-se por não adotar um método ágil de equipa, como o Scrum, por o projeto ser individual: as práticas de coordenação entre elementos não seriam aplicáveis.

Cada incremento seguiu o mesmo ciclo: (1) seleção da funcionalidade a partir dos requisitos funcionais RF1–RF7; (2) análise e desenho, incluindo as alterações necessárias ao modelo de dados; (3) implementação no backend e, em seguida, nos clientes; (4) verificação contra os critérios de aceitação definidos no capítulo 13; e (5) integração no sistema existente. O código foi versionado com Git, com um commit descritivo por funcionalidade ou correção, pelo que o histórico do repositório documenta a sequência de incrementos e assegura a rastreabilidade das alterações.

Reconhece-se que a aplicação do modelo foi informal: não houve iterações de duração fixa nem um registo formal de incrementos além do histórico de commits.

7. Manual de Utilizador

7.1. Introdução

Este manual descreve, passo a passo, como utilizar o BoxToCar. Destina-se a qualquer utilizador e não exige conhecimentos técnicos. O sistema tem dois perfis: o Gestor, que administra o armazém através do backoffice web, e o Operador, que executa as recolhas através da aplicação móvel. As figuras referidas ao longo do manual correspondem às apresentadas nos capítulos anteriores do relatório.

7.2. Guia do Gestor (Backoffice Web)

7.2.1. Criar conta e armazém

1. No ecrã de acesso, clique em "Regista-te aqui".
2. Preencha o seu nome, email e palavra-passe.
3. Indique o nome do armazém (e, opcionalmente, a cidade) e clique em "CRIAR CONTA". O armazém é criado e associado à sua conta de Gestor.

7.2.2. Iniciar sessão

4. No ecrã de acesso (Figura 3), introduza o email e a palavra-passe e clique em "ENTRAR NO SISTEMA".

7.2.3. O painel e o mapa do armazém

O painel (Figura 4) apresenta ao centro o mapa em grelha: cada quadrado é uma posição, as prateleiras estão a azul e o robô (□) desloca-se sobre o mapa. As ferramentas de gestão estão à direita e por baixo.

7.2.4. Definir o tamanho e construir prateleiras

5. Em "Expandir Chão", defina o número de colunas (Max X) e de linhas (Max Y).
6. Em "Construir Prateleira", indique a posição (X, Y) e os níveis e clique em "CRIAR ESTRUTURA".

Nota: Em alternativa, pode carregar todo o layout de uma só vez importando uma planta em ficheiro JSON, no cartão "Importar Planta".

7.2.5. Dar entrada de produtos

7. Em "Entrada de Produto", escreva o nome/SKU e a quantidade.
8. Indique a posição da prateleira (X, Y) e o nível e clique em "ARRUMAR NO STOCK" (Figura 5). A prateleira passa a mostrar o número de artigos (□).

7.2.6. Gerar ordens de picking

9. Em "Simulador ERP", escreva o produto e a quantidade e clique em "+" para o adicionar ao carrinho (pode remover no "X").
10. Clique em "GERAR ORDEM DE PICKING". O sistema valida o stock e cria as tarefas.

Nota: Se tentar encomendar mais do que existe (contando com as encomendas pendentes), a operação é bloqueada.

7.2.7. Planear a coordenação humano-máquina

11. Clique numa prateleira com produto e, no detalhe, clique em "PLANEAR COORDENAÇÃO".
12. No mapa surgem o ponto de encontro (M), o depósito (D), a rota do robô (vermelho) e a do operador (verde), e um painel com as métricas de poupança (Figura 7).

Nota: A visualização mantém-se até fechar no "X" do painel ou até realizar uma nova ação (gerar ordem, criar prateleira, etc.).

7.3. Guia do Operador (Aplicação Móvel)

7.3.1. Criar conta e iniciar sessão

13. No ecrã de início de sessão, toque em "Não tens conta? Cria uma aqui".
14. Preencha os dados, escolha o armazém na lista e toque em "CRIAR CONTA".
15. Para entrar, introduza o email e a palavra-passe e toque em "ENTRAR".

Nota: A aplicação mantém a sessão iniciada entre utilizações; só terá de entrar de novo se terminar sessão ou se esta expirar.

7.3.2. Executar uma recolha

16. Consulte a lista de tarefas pendentes; cada uma indica o produto e a localização (Figura 8).
17. Numa tarefa, toque em "PONTO DE ENCONTRO (ROBÔ)" para ver onde entregar a carga ao robô e a poupança de esforço.
18. Recolha o artigo na prateleira, leve-o até ao ponto de encontro e entregue-o ao robô.
19. Toque em "CONFIRMAR RECOLHA". A tarefa é concluída e o stock é atualizado.

7.4. Resolução de Problemas (FAQ)

Não consigo iniciar sessão. Verifique o email e a palavra-passe e confirme que o servidor está ligado.

A minha sessão expirou. Por segurança, a sessão tem duração limitada; basta iniciar sessão novamente.

Não consigo encomendar um produto. O sistema impede encomendar mais unidades do que as disponíveis; verifique o stock ou dê entrada de mais unidades.

A coordenação não aparece no mapa. Confirme que clicou em "PLANEAR COORDENAÇÃO" a partir de uma prateleira e, se necessário, recarregue a página (Ctrl+Shift+R).

8. Autenticação, Autorização e Multi-armazém

O início de sessão valida as credenciais (comparando a palavra-passe com o hash bcrypt) e emite um token JWT com validade de oito horas, que incorpora a identidade, o papel e o armazém do utilizador (Figura 3). Um middleware valida o token em todas as rotas sensíveis e disponibiliza os dados do utilizador; o armazém é sempre obtido do token, e nunca de parâmetros do cliente, garantindo o isolamento de dados. Foram implementados dois papéis: Gestor (backoffice web) e Operador (aplicação móvel).

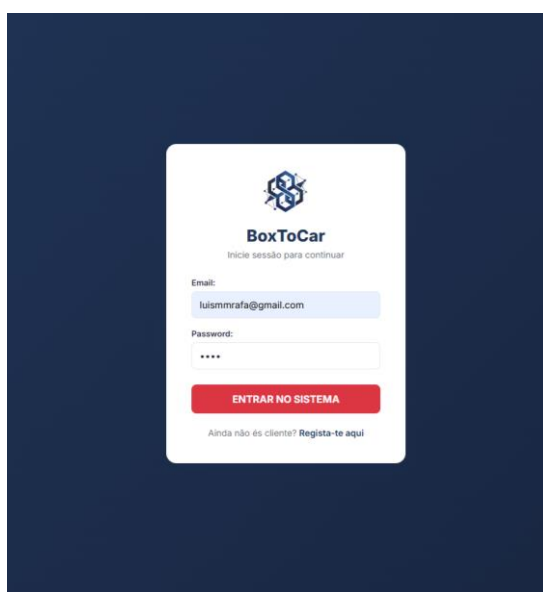


Figura 3 - Ecrã de início de sessão do backoffice web.

Numa primeira versão, o armazém era indicado pelo cliente, o que permitiria, em teoria, aceder a dados de outro armazém; a correção passou por derivar o armazém exclusivamente do token e por validar a posse dos recursos nas operações de remoção e conclusão. Os testes (capítulo 13) confirmaram que um utilizador não acede nem altera dados de outro armazém e que as rotas sensíveis rejeitam pedidos sem token válido.

9. Gestão do Armazém e Mapa Interativo

O backoffice apresenta o armazém como uma grelha 2D, em que cada célula é uma posição (Figura 4). As prateleiras são assinaladas e, ao serem seleccionadas, mostram os produtos que contêm. As dimensões da grelha são configuráveis e ajustam-se à distribuição das prateleiras.



Figura 4 - Backoffice web com o mapa interativo do armazém.

As prateleiras podem ser criadas individualmente ou importadas em conjunto a partir de um ficheiro de planta em formato JSON (RF2), o que concretiza a importação dinâmica de layouts prevista na proposta. A importação substitui o layout anterior numa transação, evitando sobreposições. Esta grelha constitui o espaço sobre o qual operam os algoritmos de rota e de coordenação descritos no capítulo 11.

10. Gestão de Stock e Ordens de Picking

O gestor dá entrada de produtos indicando o nome, a posição e a quantidade; cada unidade é um registo, e o nível de stock corresponde ao número de registos. As ordens de picking são geradas por um simulador que reproduz a chegada de encomendas de um sistema ERP (Figura 5).



A interface é dividida em duas colunas. A coluna da esquerda, intitulada 'Entrada de Produto', contém campos para 'Nome / SKU:' (com o valor 'monitor'), 'Qtd:' (com o valor '2'), 'Ir ao X:' (com o valor '5'), 'Ir ao Y:' (com o valor '2') e 'Nível:' (com o valor '1'). Abaixo destes campos há um botão verde com o texto 'ARRUMAR NO STOCK'. A coluna da direita, intitulada 'Simulador ERP (Múltiplos Itens)', contém um campo 'Produto' com o valor '1' e um botão verde com o símbolo '+'. Abaixo disso, há duas linhas de confirmação: '✓ 1x monitor' e '✓ 2x rato', cada uma com um botão vermelho com o símbolo 'x'. No fundo da coluna da direita, há um botão vermelho com o texto 'GERAR ORDEM DE PICKING'.

Figura 5 - Entrada de produtos e simulador de ordens de picking (ERP).

Antes de criar cada ordem, o sistema valida a disponibilidade, calculando o stock disponível como a diferença entre o stock físico e as quantidades já comprometidas em encomendas pendentes, o que impede a sobre-alocação. Ao concluir uma tarefa, o operador desencadeia a baixa de stock; todas as operações que envolvem várias alterações ocorrem dentro de transações. A Figura 6 resume o fluxo do processo.

Figura — Fluxo do processo de picking

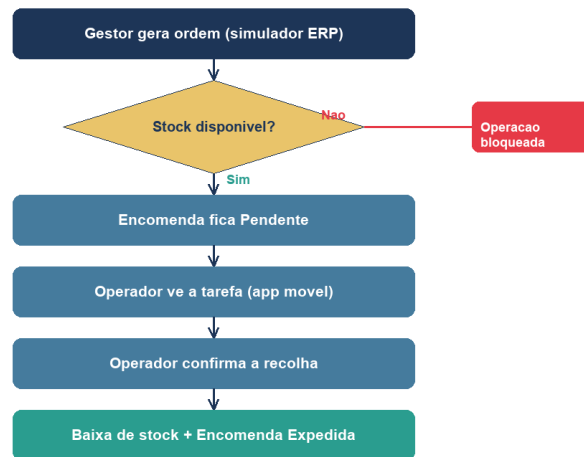


Figura 6 - Fluxo do processo de picking.

11. Coordenação Humano-Máquina

Este capítulo descreve o núcleo do projeto e a principal contribuição face a um WMS convencional: a coordenação lógica entre o operador humano e o robô virtual, que concretiza o paradigma de picking colaborativo da Indústria 5.0.

11.1. Modelo de coordenação

O armazém é modelado como uma grelha 2D em que as prateleiras são obstáculos e as restantes células são corredores livres. O robô circula exclusivamente pelo corredor principal (o perímetro da grelha), partindo de um ponto de expedição (o depósito). Para cada recolha, o operador dirige-se à prateleira, recolhe o artigo e transporta-o apenas até ao ponto de encontro — a célula do corredor principal mais próxima da prateleira —, onde o entrega ao robô. O robô assume então o transporte pesado até à expedição, libertando o operador para a tarefa seguinte.

11.2. Cálculo do ponto de encontro

O ponto de encontro é determinado de modo a minimizar a distância percorrida pelo operador com carga. Para tal, executa-se uma procura em largura (BFS) multi-origem a partir das células de acesso à prateleira (os seus vizinhos livres), obtendo-se a distância de qualquer célula livre até à recolha. O ponto de encontro é a célula do corredor principal com menor distância assim obtida. A rota do robô resulta de uma segunda BFS, a partir do depósito, e a rota do operador com carga da reconstrução do caminho até ao ponto de encontro.

11.3. Algoritmo

O algoritmo de coordenação pode ser resumido no seguinte pseudocódigo (a implementação completa consta do Apêndice C):

```
acessos <- vizinhos livres da prateleira alvo
distA <- BFS(origens = acessos) // dist. a partir da recolha
encontro <- celula do perimetro que minimiza distA
rotaRobo <- BFS(deposito) -> caminho ate encontro
rotaOper <- caminho de acessos ate encontro (via distA)
poupanca <- distA[deposito] - distA[encontro] // esforco poupado
```

O algoritmo executa duas procuras em largura (a partir da prateleira e a partir do depósito) e uma varredura do perímetro; como cada uma percorre a grelha uma vez, a complexidade mantém-se linear, $O(L \times C)$. Na prática, cada plano foi calculado em menos de 1 ms nos layouts testados (cerca de 0,02 ms numa grelha 7×6), o que assegura um cálculo em tempo interativo.

11.4. Implementação

A lógica foi isolada num módulo próprio do backend (coordenacao.js), independente da base de dados e, por isso, testável de forma unitária. Um endpoint da API recebe a prateleira-alvo (por posição ou por produto), carrega a planta do armazém, invoca o módulo e devolve o ponto de encontro, as duas rotas e as métricas de eficiência. O excerto seguinte ilustra o cerne do cálculo do ponto de encontro:

```
for (const [cel, d] of distAcesso)
  if (ehCorredorPrincipal(cel) && d < melhor) {
    melhor = d; pontoEncontro = cel;
  }
```

11.5. Visualização

O backoffice web permite visualizar, sobre o mapa, o ponto de encontro, o depósito e as rotas do robô e do operador, acompanhados das métricas de eficiência (Figura 7). A aplicação móvel apresenta ao operador o ponto de encontro e a poupança de esforço correspondente (Figura 8).

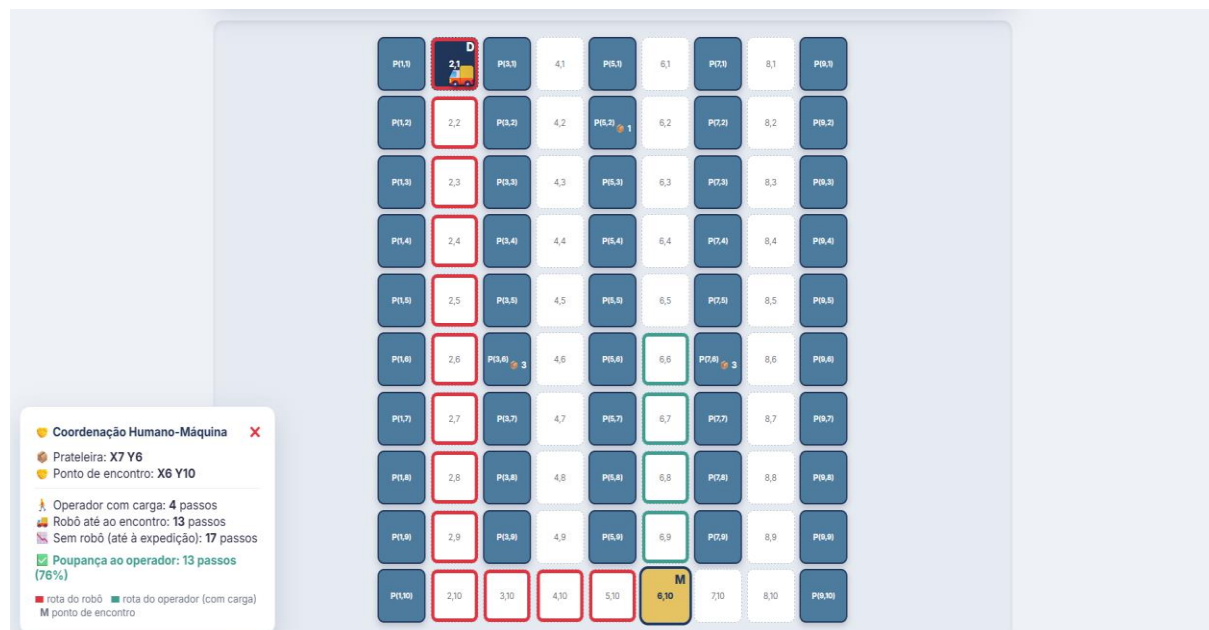


Figura 7 - Coordenação humano-máquina: ponto de encontro e rotas calculadas.

11.6. Métricas de eficiência

Para cada recolha, o sistema quantifica três grandezas: a distância percorrida pelo operador com carga (até ao ponto de encontro), a distância percorrida pelo robô (do depósito ao ponto de encontro) e a poupança — a distância que o operador percorreria com carga caso tivesse de transportar o artigo até à expedição sem o apoio do robô. Estas métricas, além de tornarem visível o valor da coordenação, servem de base à validação quantitativa apresentada no capítulo 13.

12. Aplicação Móvel do Operador

A aplicação móvel, desenvolvida em Flutter, permite ao operador iniciar sessão, consultar as tarefas pendentes do seu armazém e, para cada uma, consultar o ponto de encontro com o robô e a poupança de esforço associada (Figura 8), antes de confirmar a recolha e desencadear a baixa de stock. O token de autenticação é guardado no dispositivo, mantendo a sessão ativa entre utilizações.

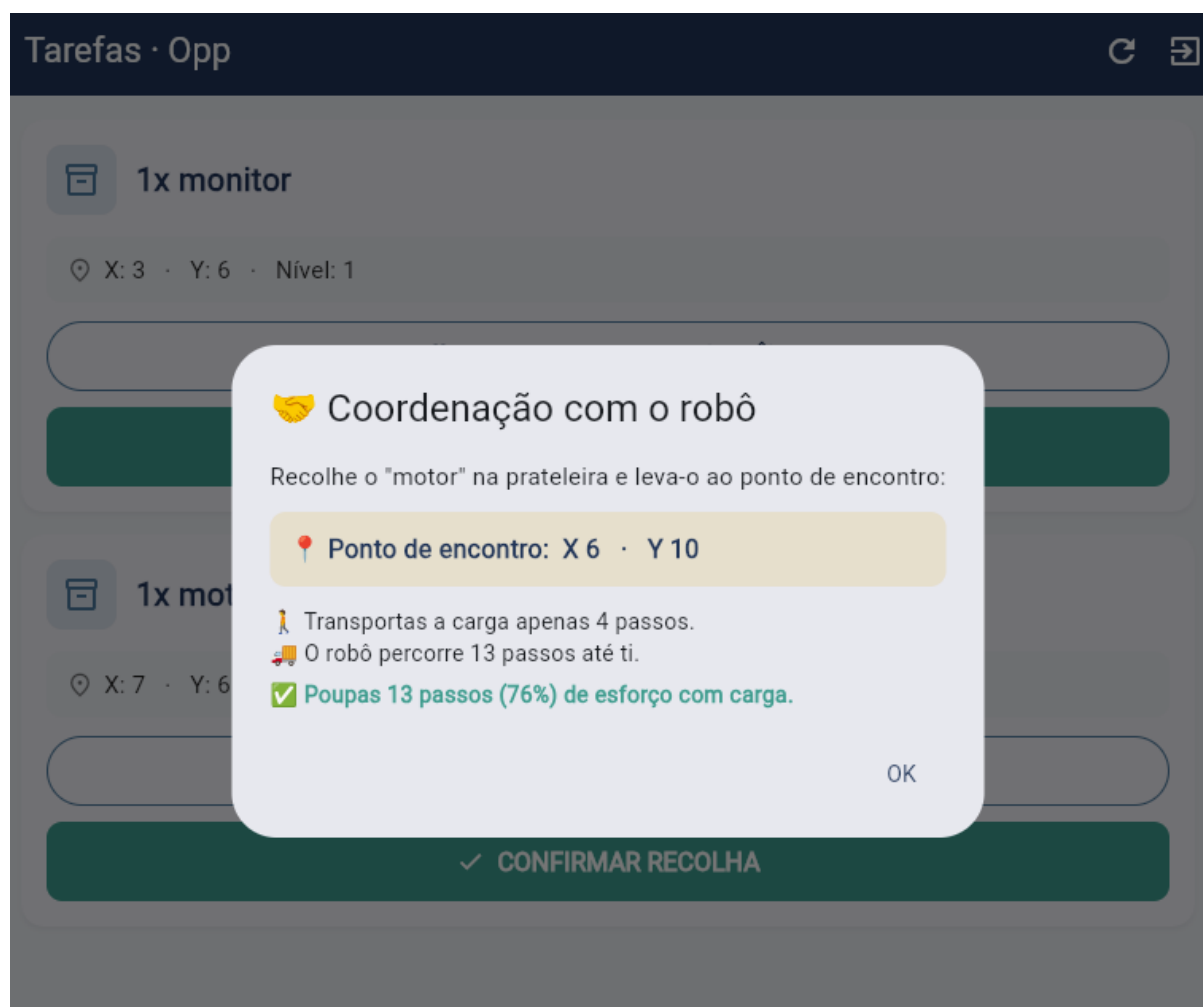


Figura 8 - Aplicação móvel do operador: tarefa e ponto de encontro com o robô.

A comunicação com a API é feita por pedidos HTTP com o token no cabeçalho de autorização; perante uma sessão expirada, a aplicação limpa o token e regressa ao início de sessão. A interface partilha a identidade visual do backoffice, garantindo consistência entre as duas vertentes do sistema.

13. Testes de Validação

13.1. Metodologia de teste

A validação seguiu uma abordagem de caixa-preta, exercitando o sistema através da sua API REST, de forma independente das interfaces gráficas. Os testes foram executados num ambiente controlado (Node.js com base de dados MySQL local), sobre um armazém de referência de dimensão 7×6, com corredores definidos, e recorrendo a um cliente automatizado desenvolvido em Python (biblioteca urllib), que emite os pedidos e verifica automaticamente as respostas. Distinguiram-se dois grupos de testes: testes funcionais, que verificam o comportamento das operações críticas face a resultados esperados; e testes ao algoritmo de coordenação, que medem quantitativamente a eficiência dos pontos de encontro calculados. O critério de aceitação foi, para os primeiros, a conformidade com o resultado esperado e, para os segundos, a obtenção de rotas válidas e de poupança não negativa em todos os cenários.

13.2. Testes funcionais

A Tabela 4 sintetiza os principais casos de teste funcionais, com o objetivo, a ação, o resultado esperado e o resultado obtido.

ID	Objetivo / ação	Resultado esperado	Obtido
T1	Aceder a rota sensível sem token	Rejeição (HTTP 401)	Conforme
T2	Aceder a dados de outro armazém	Acesso negado / isolado	Conforme
T3	Encomendar acima do stock disponível	Operação bloqueada (400)	Conforme
T4	Concluir tarefa de picking	Baixa de stock e estado atualizado	Conforme
T5	Importar nova planta (JSON)	Layout anterior substituído	Conforme
T6	Plano de coordenação de produto inexistente	Erro tratado (404)	Conforme

Tabela 4 - Casos de teste funcionais e resultados obtidos.

13.3. Testes ao algoritmo de coordenação

Para avaliar quantitativamente a coordenação, calcularam-se os planos de recolha para um conjunto de prateleiras representativas do armazém de referência, registando-se a distância do operador com carga, a distância do robô e a poupança de esforço (Tabela 5). A poupança percentual traduz a fração da distância com carga que é evitada ao operador graças ao robô.

Prateleira	Ponto de encontro	Operador c/ carga	Robô	Poupança	Poupança %
(5,3)	(6,1)	2 passos	9 passos	3 passos	60%

Prateleira	Ponto de encontro	Operador c/ carga	Robô	Poupança	Poupança %
(5,5)	(6,6)	1 passo	8 passos	4 passos	80%
(4,5)	(3,6)	1 passo	5 passos	3 passos	75%
(2,3)	(1,4)	1 passo	5 passos	1 passo	50%
(5,6)	(6,6)	0 passos	8 passos	8 passos	100%

Tabela 5 - Resultados do algoritmo de coordenação (armazém 7×6).

Para verificar que o algoritmo generaliza a diferentes plantas, repetiu-se a avaliação num segundo layout, de dimensão 5×7 e com uma disposição distinta das prateleiras e um corredor transversal (Tabela 6). Os resultados mantêm o mesmo padrão, com poupanças entre 80% e 100%, confirmando que o comportamento não depende do layout específico usado na Tabela 5.

Prateleira	Ponto de encontro	Operador c/ carga	Robô	Poupança	Poupança %
(5,3)	(5,4)	0 passos	6 passos	6 passos	100%
(1,6)	(2,7)	1 passo	6 passos	4 passos	80%
(3,7)	(2,7)	0 passos	6 passos	6 passos	100%

Tabela 6 - Resultados do algoritmo de coordenação num segundo layout (5×7).

13.4. Testes unitários ao módulo de coordenação

Como o módulo de coordenação (coordenacao.js) é independente da base de dados, foi também testado de forma unitária, invocando diretamente a função de planeamento sobre grelhas construídas em código. Para cada cenário, verificou-se que o ponto de encontro pertence ao corredor principal, que as rotas do operador e do robô são contíguas e não atravessam prateleiras, e que a poupança calculada é coerente com as distâncias. Estes testes unitários complementam os testes de integração pela API (secções 13.2 e 13.3), cobrindo isoladamente a lógica do algoritmo.

13.5. Análise dos resultados

Os testes funcionais confirmaram o comportamento esperado em todos os casos, validando a segurança, o isolamento entre armazéns e a correção do controlo de stock e do fluxo de picking. Os testes ao algoritmo de coordenação demonstraram, em todos os cenários, rotas válidas e poupança não negativa, com uma redução média da distância percorrida pelo operador com carga de aproximadamente 73% e um máximo de 100% (quando a prateleira é adjacente ao corredor principal). Estes resultados evidenciam, de forma mensurável, o benefício central do sistema: transferir o esforço de transporte do operador para o robô, em conformidade com o objetivo da proposta.

14. Conclusões

14.1. Resultados obtidos

O projeto cumpriu os objetivos definidos na proposta, resultando no BoxToCar — uma prova de conceito funcional de picking colaborativo humano-máquina. Foram concretizadas as três componentes (API, backoffice web e aplicação móvel), a importação dinâmica de layouts, e, sobretudo, o sistema de coordenação que calcula pontos de encontro eficientes por algoritmo exato, com métricas que comprovam a redução do esforço do operador.

14.2. Dificuldades e soluções

As principais dificuldades incluíram a definição de um modelo de coordenação simultaneamente realista e computável — resolvida restringindo o robô ao corredor principal e reduzindo o problema a um caso do ponto de encontro sobre grelha —, o reforço iterativo da segurança para garantir o isolamento entre armazéns, e a compatibilização do código com o esquema da base de dados. Cada dificuldade foi ultrapassada e validada por testes.

14.3. Melhorias futuras

- Substituir a BFS por A* com custos diferenciados (corredores rápidos/lentos) e otimizar sequências de várias recolhas (routing multi-tarefa).
- Coordenar múltiplos operadores e robôs em simultâneo, com afetação dinâmica de tarefas.
- Suportar a importação de layouts em XML, além de JSON, e integrar dados reais de um ERP.
- Introduzir indicadores de desempenho (distância total, tempo, ocupação do robô) e relatórios.

14.4. Limitações

Sendo uma prova de conceito, o BoxToCar apresenta limitações que importa assumir. O robô é virtual, único e circula em rotas fixas, não modelando a navegação autónoma nem a prevenção de colisões em tempo real de múltiplos robôs. O modelo de dados adota simplificações: cada unidade de stock é um registo distinto (o que não escala para grandes volumes), as encomendas ligam-se ao produto pelo nome (e não por chave estrangeira) e as configurações assumem um único armazém por gestor. Do lado da segurança, embora as consultas sejam parametrizadas, uma cláusula LIMIT recorre a um valor convertido para inteiro em vez de parametrização direta. Por fim, a validação quantitativa incidiu sobre layouts de pequena dimensão. Estas limitações

não comprometem a demonstração do conceito e orientam o trabalho futuro (secção 14.3).

14.5. Contributo pessoal e aprendizagens

Este projeto permitiu integrar, num sistema completo, conhecimentos de desenvolvimento web e móvel, bases de dados, segurança e algoritmos, e ainda aprofundar um tema atual — a colaboração humano-máquina da Indústria 5.0. O desenho e a implementação do algoritmo de coordenação, bem como a sua validação quantitativa, constituíram a componente mais desafiante e enriquecedora, com um contributo determinante para a minha formação enquanto engenheiro informático.

Referências Bibliográficas

- Azadeh, K., De Koster, R., & Roy, D. (2019). Robotized and automated warehouse systems: Review and recent developments. *Transportation Science*, 53(4), 917–945. <https://doi.org/10.1287/trsc.2018.0873>
- Boysen, N., De Koster, R., & Weidinger, F. (2019). Warehousing in the e-commerce era: A survey. *European Journal of Operational Research*, 277(2), 396–411. <https://doi.org/10.1016/j.ejor.2018.08.023>
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to algorithms* (3.^a ed.). MIT Press.
- Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tese de doutoramento, University of California, Irvine].
- Fragapane, G., De Koster, R., Sgarbossa, F., & Strandhagen, J. O. (2021). Planning and control of autonomous mobile robots for intralogistics: Literature review and research agenda. *European Journal of Operational Research*, 294(2), 405–426. <https://doi.org/10.1016/j.ejor.2021.01.019>
- Google. (2023). *Flutter documentation*. <https://docs.flutter.dev>
- Gualtieri, L., Rauch, E., & Vidoni, R. (2021). Emerging research fields in safety and ergonomics in industrial collaborative robotics: A systematic literature review. *Robotics and Computer-Integrated Manufacturing*, 67, Article 101998. <https://doi.org/10.1016/j.rcim.2020.101998>
- Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107. <https://doi.org/10.1109/TSSC.1968.300136>
- Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. <https://doi.org/10.17487/RFC7519>
- Larman, C., & Basili, V. R. (2003). Iterative and incremental development: A brief history. *Computer*, 36(6), 47–56. <https://doi.org/10.1109/MC.2003.1204375>
- Leng, J., Sha, W., Wang, B., Zheng, P., Zhuang, C., Liu, Q., Wuest, T., Mourtzis, D., & Wang, L. (2022). Industry 5.0: Prospect and retrospect. *Journal of Manufacturing Systems*, 65, 279–295. <https://doi.org/10.1016/j.jmsy.2022.09.017>
- Maddikunta, P. K. R., Pham, Q. V., Prabadevi, B., Deepa, N., Dev, K., Gadekallu, T. R., Ruby, R., & Liyanage, M. (2022). Industry 5.0: A survey on enabling technologies and potential applications. *Journal of Industrial Information Integration*, 26, Article 100257. <https://doi.org/10.1016/j.jii.2021.100257>

- Provos, N., & Mazières, D. (1999). A future-adaptable password scheme. In *Proceedings of the USENIX Annual Technical Conference* (pp. 81–91). USENIX Association.
- Sommerville, I. (2016). *Software engineering* (10.^a ed.). Pearson.
- Van Gils, T., Ramaekers, K., Caris, A., & de Koster, R. (2018). Designing efficient order picking systems by combining planning problems: State-of-the-art classification and review. *European Journal of Operational Research*, 267(1), 1–15. <https://doi.org/10.1016/j.ejor.2017.09.002>
- Xu, X., Lu, Y., Vogel-Heuser, B., & Wang, L. (2021). Industry 4.0 and Industry 5.0—Inception, conception and perception. *Journal of Manufacturing Systems*, 61, 530–535. <https://doi.org/10.1016/j.jmsy.2021.10.006>

Glossário

AMR — Robô móvel autónomo capaz de se deslocar num espaço para transportar mercadoria; neste projeto, modelado de forma virtual.

Ponto de encontro — Célula do corredor principal onde o operador entrega a carga ao robô, minimizando o transporte manual.

Picking — Processo de recolha física dos produtos de uma encomenda a partir das suas localizações.

Rendezvous — Problema de determinar um ponto de convergência entre dois ou mais agentes com origens distintas.

BFS — Algoritmo de procura em largura que determina o caminho mais curto em número de passos numa grelha uniforme.

Token — Credencial digital assinada, emitida no início de sessão, que identifica e autoriza o utilizador.

WMS — Sistema informático de gestão das operações de um armazém.

Apêndices

Apêndice A — Estrutura de rotas da API

POST	/api/registo	(publico)	cria armazem + gestor
POST	/api/operador/registo	(publico)	cria operador num armazem
POST	/api/login	(publico)	devolve token JWT
GET	/api/inventario	(token)	prateleiras e produtos
POST	/api/produtos/novo	(token)	entrada de produto
POST	/api/prateleiras/nova	(token)	cria prateleira
GET	/api/tarefas/pendentes	(token)	tarefas do operador
POST	/api/operador/concluir	(token)	confirma recolha
POST	/api/encomendas/carrinho	(token)	gera ordens de picking
POST	/api/armazem/importar	(token)	importa planta (JSON)
GET	/api/coordenacao/plano	(token)	ponto de encontro + rotas

Apêndice B — Esquema das tabelas da base de dados

```
Armazens(ID, Nome, Cidade)
Usuarios(ID, Nome, Email, Senha, Cargo, ArmazemID)
Prateleiras(ID, PosX, PosY, Niveis, ArmazemID)
Produtos(ID, Nome, PrateleiraID, Nivel, ArmazemID)
Encomendas(ID, ProdutoNome, Quantidade, Estado, Data, ArmazemID)
Configuracoes(Chave, Valor)
```

Apêndice C — Pseudocódigo do algoritmo de coordenação

```
função planejarRecolha(prateleiras, grelha, alvo, deposito):
    ocupadas <- {celulas das prateleiras}
    acessos <- vizinhos livres de alvo
    distA <- BFS(origens = acessos, ocupadas)
    encontro <- argmin_{c in perimetro livre} distA[c]
    distDep <- BFS(origem = deposito, ocupadas)
    rotaRobo <- caminho(distDep, encontro)
    rotaOper <- caminho(distA, encontro)
    poupanca <- distA[deposito] - distA[encontro]
    devolver {encontro, rotaRobo, rotaOper, metricas}
```

Apêndice D — Declaração de utilização de ferramentas de inteligência artificial

Apresenta-se em seguida o detalhe da utilização de ferramentas de inteligência artificial no desenvolvimento deste projeto, conforme declarado na secção 1.4. Os prompts são transcritos tal como foram submetidos.

D.1 — Aprendizagem de Git e GitHub

Ferramenta e versão: Claude Code, versão 2.1.190

Objetivo: Compreender os conceitos de controlo de versões e o fluxo de trabalho do Git e do GitHub, sem experiência prévia na ferramenta.

Prompt:

Atua como um professor de Engenharia de Software. Sou novo no uso de controlo de versões e preciso de um guia para iniciantes, passo a passo, sobre como utilizar o GitHub.

Por favor, explica-me:

- 1. Os conceitos básicos: o que é um repositório, um commit e um push.*
- 2. Como usar a aplicação oficial (GitHub Desktop) para conectar o meu código local ao repositório na nuvem, desde criar o repositório até enviar o código.*
- 3. Como mandar o código para lá através do terminal (comandos básicos de Git), caso eu prefira não usar a aplicação visual.*

Usa uma linguagem simples, dá exemplos práticos e foca-te no fluxo de trabalho diário mais comum (alterar código -> guardar -> enviar para o GitHub).

Resultado obtido: Explicação dos conceitos de repositório, commit e push, e do fluxo de trabalho diário, tanto pela aplicação GitHub Desktop como pelo terminal.

Verificação do autor: Os comandos foram executados pelo autor e o resultado confirmado diretamente no repositório remoto.

D.2 — Automatização de commits e envio para o repositório

Ferramenta e versão: Claude Code, versão 2.1.190

Objetivo: Gerar mensagens de commit descritivas, segundo o padrão Conventional Commits, e enviar as alterações para o repositório remoto.

Prompt:

A tua função é atuar como um Agente de Integração de Código.

Recebeste as alterações de código mais recentes.

Executa as seguintes tarefas automaticamente:

- 1. Análise de Código: Verifica as alterações introduzidas para garantir que não existem erros críticos de sintaxe que quebrem a compilação.*
- 2. Geração de Commit Semântico: Analisa o diff das alterações e gera automaticamente uma mensagem de commit seguindo o padrão de Conventional Commits (ex: `feat: adiciona login`, `fix: corrige erro de base*

de dados`, `refactor: otimiza função`). A mensagem deve ser em português, clara e descritiva.

3. Push Direto: Executa o commit com a mensagem gerada e faz o push imediato destas alterações para o repositório remoto na branch atual.

Condição de Falha: Se o código contiver erros óbvios que inviabilizem a execução do sistema, interrompe o processo, não faças o commit nem o push, e regista o erro no log do pipeline.

Resultado obtido: Mensagens de commit geradas automaticamente em português. A ferramenta acrescenta, por defeito, uma atribuição de coautoria (Co-Authored-By: Claude) que fica registada no histórico público do repositório.

Verificação do autor: Embora o prompt determinasse o envio imediato, a ferramenta solicita autorização antes de executar cada comando. O autor verificou a mensagem de commit gerada e as alterações correspondentes antes de autorizar o commit e o envio para o repositório remoto.

D.3 — Revisão crítica do relatório

Ferramenta e versão: Claude (claude.ai), modelo Opus 4.8

Objetivo: Obter uma revisão crítica do relatório quanto ao rigor académico, à consistência técnica e à estrutura.

Prompt:

Atua como um Professor Universitário de Engenharia Informática e um Senior Software Engineer. O teu objetivo é analisar criticamente o meu relatório final de projeto de licenciatura, intitulado 'BoxToCar - Sistema de Gestão de Armazém com Picking Assistido'. [...] Entrega-me uma lista estruturada de Pontos Fortes e Áreas a Melhorar, dividida por capítulos. Onde encontrares problemas técnicos ou frases mal construídas, cita a minha frase original e sugere uma reescrita direta.

Resultado obtido: Lista estruturada de pontos fortes e de áreas a melhorar, por capítulo, com identificação de erros de numeração de figuras, de inconsistências no modelo de dados e de lacunas no capítulo de testes.

Verificação do autor: Cada sugestão foi avaliada pelo autor e aplicada seletivamente. Algumas sugestões não foram adotadas.

D.4 — Fundamentação do referencial metodológico (secção 6.4)

Ferramenta e versão: Claude (claude.ai), modelo Opus 4.8

Objetivo: Identificar o referencial metodológico correspondente ao processo de desenvolvimento efetivamente seguido, na sequência de uma questão colocada pelo orientador.

Prompt:

[Citação do email do orientador, colada na conversa] Na página 24 afirma "O desenvolvimento seguiu uma metodologia iterativa e incremental: cada funcionalidade foi implementada, testada e integrada de forma sucessiva." Pode explicar-me em que metodologia de trabalho se baseia este processo? Qual foi o referencial metodológico adotado? Quais são os passos indicados nesta abordagem metodológica?

Resultado obtido: Identificação do modelo de desenvolvimento incremental (Sommerville, 2016) e da tradição do desenvolvimento iterativo e incremental (Larman & Basili, 2003), com a descrição dos cinco passos do ciclo de cada incremento.

Verificação do autor: As duas referências foram verificadas na fonte pelo autor antes de serem incorporadas na bibliografia.

D.5 — Formatação das referências bibliográficas

Ferramenta e versão: Claude (claude.ai), modelo Opus 4.8

Objetivo: Rever a formatação das referências bibliográficas segundo as normas APA, 7.^a edição.

Prompt:

Podes tratar das Referências bibliográficas na forma APA por favor

Resultado obtido: Dezasseis referências formatadas: itálico no nome da revista e no volume, títulos em minúscula, travessão nos intervalos de páginas e acréscimo dos onze DOI existentes.

Verificação do autor: Os DOI foram conferidos individualmente em fontes independentes. As cinco referências sem DOI (dois livros, uma tese, uma documentação técnica e um artigo em atas) foram confirmadas como tal.

D.6 — Correção da numeração das figuras

Ferramenta e versão: Claude (claude.ai), modelo Opus 4.8

Objetivo: Verificar a coerência do documento após a conversão das legendas em campos de numeração automática.

Prompt:

Ficou bem as alterações?

Resultado obtido: Detecção de que as referências às figuras no corpo do texto mantinham a numeração anterior, deixando de corresponder às legendas. Foram realinhadas catorze referências.

Verificação do autor: O documento foi revisto pelo autor após a correção.

D.7 — Página web do projeto

Ferramenta e versão: Claude (claude.ai), modelo Opus 4.8

Objetivo: Determinar a forma adequada de publicar a página web do projeto e construir a respetiva página.

Prompt:

A única dúvida que tenho é como fazer o site, se faço localhost ou se faço de outra forma.

Resultado obtido: Página informativa do projeto (ficheiro index.html), publicada através do GitHub Pages, com a identificação do projeto, a arquitetura, o algoritmo de coordenação, os resultados e as ligações para o código e para o relatório.

Verificação do autor: O conteúdo da página foi verificado pelo autor antes da publicação.